SIGSF

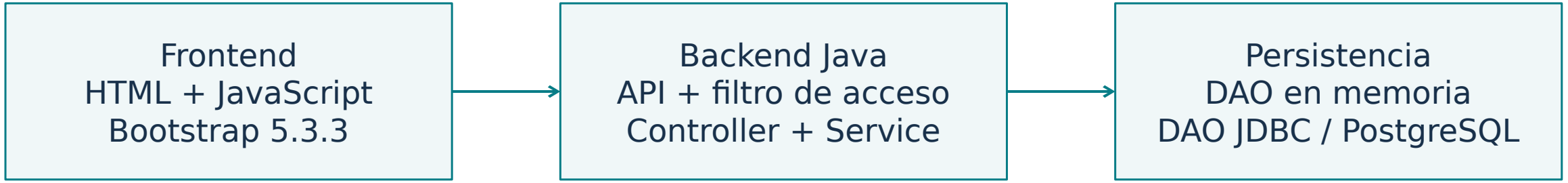
Plan de pruebas de seguridad

Sistema Integral de Gestión de Servicios Financieros

Miguel Ángel Machaca Machaca
Ricardo Gabriel Cervantes Tejada
Cesar Huaripaucar Huaman

Curso Integrador I Sistemas - Software, UTP
Docente: Rene Alonso Nieto Valencia

Alcance y entorno de evaluación



Activos y actores

Cuentas, datos de clientes, pagos y auditoría.
Roles: ADMIN, CAJERO, ANALISTA, SUPERVISOR y CLIENTE.

Condiciones de la prueba

Localhost y datos de demostración.
Java 21.0.13, código fuente/objetivo 17.
Sin conexión a billeteras reales.

Plan de pruebas de seguridad

Área	Caso y criterio de aceptación	Estado
Penetración dirigida	Token inválido, RBAC e IDOR: 401/403	Ejecutada
Entradas / fuzzing	20 vectores inválidos: 4xx y sin token	Ejecutada
Deserialización	Bytes Java rechazados; metadatos inertes	Acotada
Configuración	Cabeceras, tipo de contenido y JSON estricto	Corregida
Vulnerabilidades	npm ejecutado; ZAP y JAR por evaluar	npm: hallazgos
Base de datos	SQLi, permisos, transacciones y restauración	Pendiente en BD

Entorno aislado, pruebas repetibles y registro de cada resultado.

Criterio de salida propuesto: controles críticos aprobados, hallazgos resueltos y evidencia completa.

Herramientas elegidas

Herramienta	Uso en este proyecto	Evidencia
JUnit 4.13.2	Unitarias, API, acceso y reglas de negocio	junit.txt
Postman / Newman 6.2.1	Colección API compartible y aserciones	newman.xml
Python 3	Fuzzing acotado y verificación HTTP	entradas_seguridad.json
Playwright 1.63.0	Interfaz, XSS y flujo de pagos	14 casos definidos, pendientes
npm audit / ZAP	Dependencias npm / DAST autenticado	npm-audit.json
Git + GitHub Actions	Trazabilidad del código y regresión	Rama y commits locales

Logback conserva eventos de acceso, seguridad y errores.

Rutas oficiales en el anexo 12. Ejecución unificada: python scripts/validar_proyecto.py.

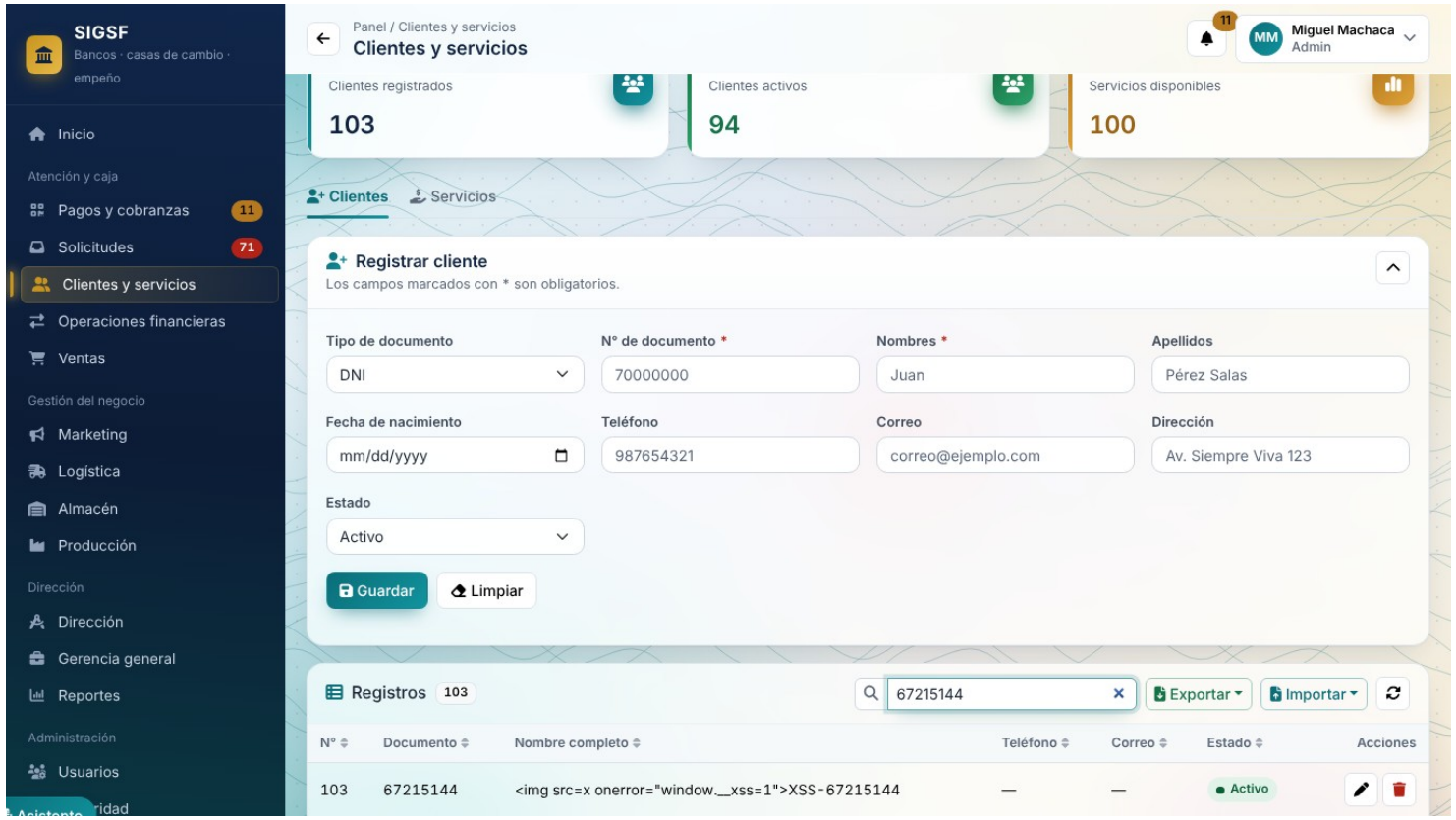
Resultados obtenidos

Bloque	Resultado	Interpretación
JUnit, actual	214 / 214 aprobadas	22 pruebas nuevas integradas
Newman, actual	26 peticiones, 63 aserciones 0 fallos	API reproducida en memoria
Pruebas adicionales	24 / 24 cumplen el criterio	JSON-SUF y CFG-TYPE corregidos
Playwright, actual	14 casos definidos	Sin reejecutar por restricción local
npm audit	21 paquetes señalados	1 crítico, 13 altos y 7 moderados

Pendientes: corregir alertas npm; evaluar PostgreSQL, ZAP y JAR.

Alertas npm de herramientas de pruebas; la explotación en la aplicación no está demostrada.

Evidencia de interfaz y código



XSS como texto

Regresión JSON-SUF

```
rechazado(
 enviar(LOGIN + " trailing",
 "application/json"), 400);
```

Ahora: HTTP 400 con credenciales válidas.

Izquierda: captura heredada 04/11/2026. Derecha: regresión ejecutada en v5.6.0 el 10/11/2026.

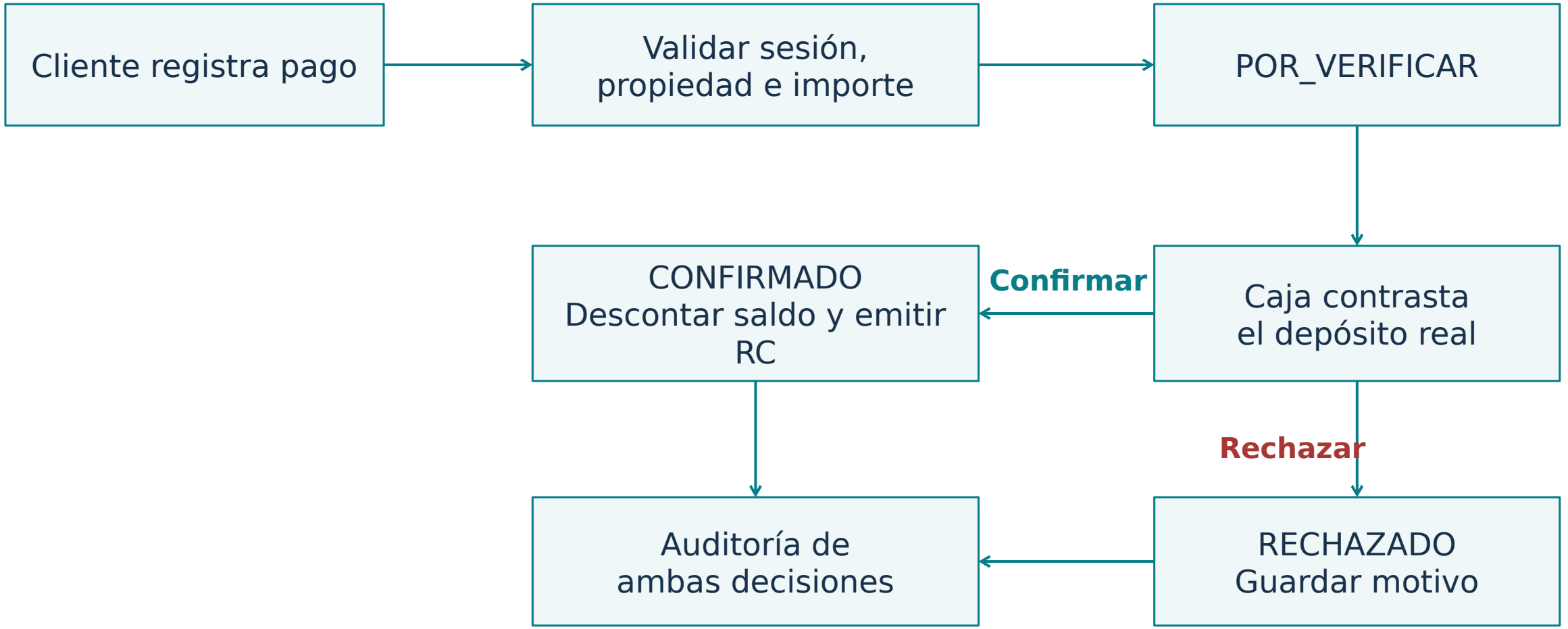
Controles integrados y validación pendiente

Control	Implementación	Verificación
JSON estricto	Fin de documento, claves y profundidad	Pruebas unitarias y HTTP aprobadas
Tipo de contenido	application/json y UTF-8	HTTP 415 para tipo no admitido
Política de scripts	26 scripts externos y eventos explícitos	CSP estricta y referencias comprobadas
Validación ampliada	npm ejecutado; JDBC/ZAP/JAR pendientes	21 alertas npm por corregir

Una versión integrada del proyecto

Aplicación ejecutable, código, pruebas, modelos, diccionario y documentación.

Anexo 1. Proceso de pago



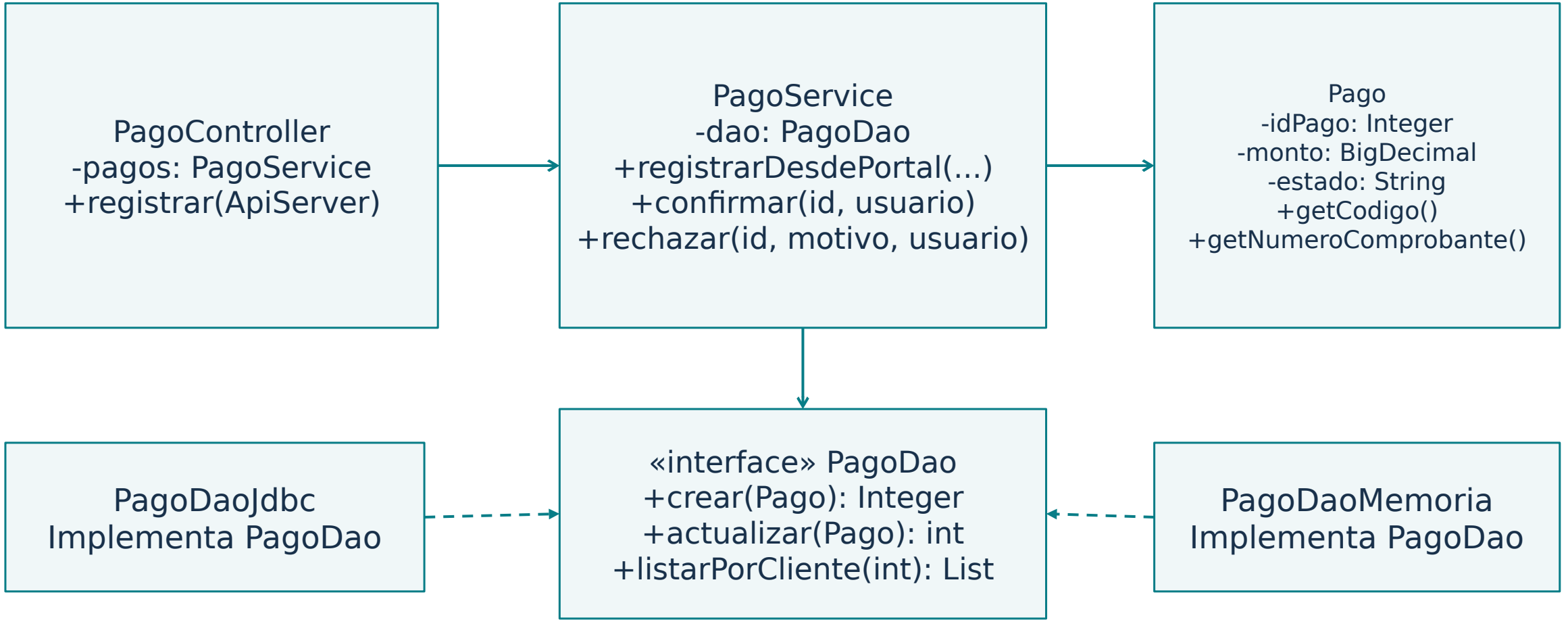
Proceso reconstruido desde PagoService.java. La verificación de la billetera es manual.

Anexo 2. MVC, DTO, DAO y SOLID

Elemento	Evidencia	Propósito
MVC	web/ + controller/ + service/ y model/	Separar interfaz y lógica
DTO	LoginDTO, UsuarioDTO, SesionDTO	Transferir datos específicos
DAO	PagoDao, PagoDaoJdbc, PagoDaoMemoria	Aislar persistencia
SRP / DIP	Controller, Service y DAO inyectado	Responsabilidades e interfaces
OCP / ISP	DaoFactory; LecturaDao y EscrituraDao	Extensión e interfaces acotadas
LSP	Implementaciones JDBC y memoria	Sustitución diseñada; falta prueba en BD

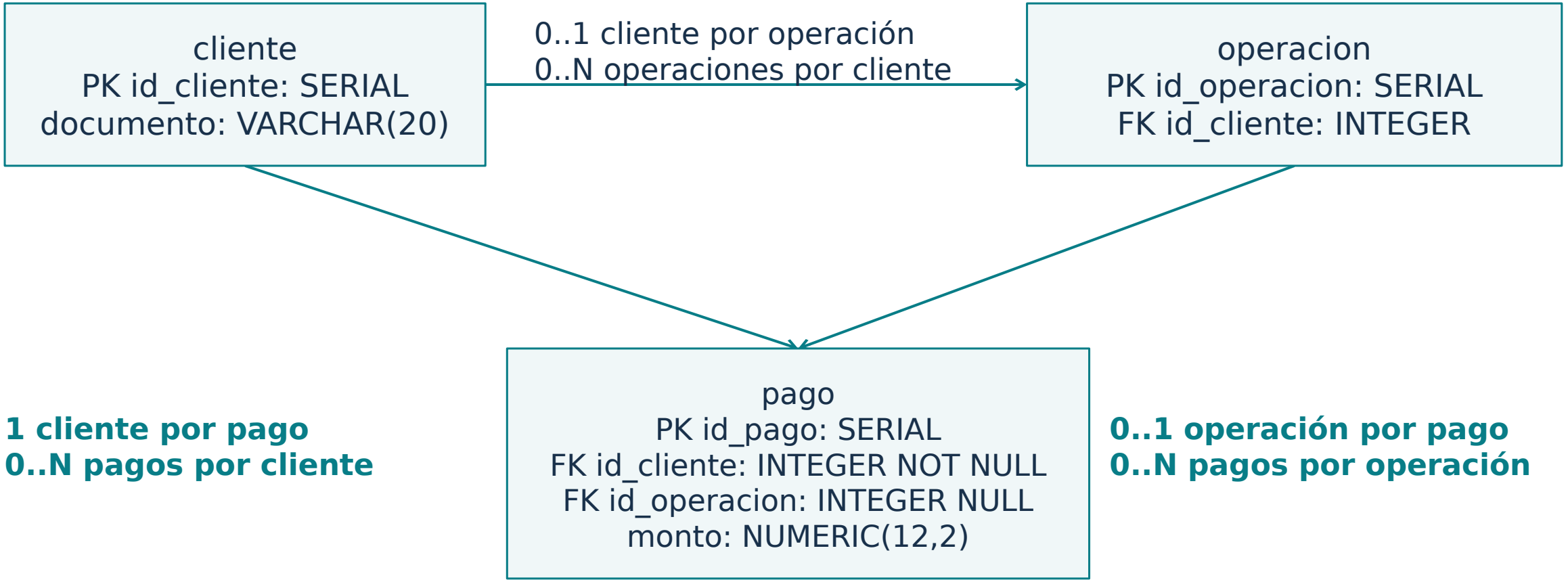
Los patrones se identifican por clases y dependencias observadas en src/.

Anexo 3. Modelo de clases de pagos



Extracto del modelo. Atributos y métodos reales; las dependencias restantes están omitidas por legibilidad.

Anexo 4. Modelo lógico y físico



1 cliente por pago
0..N pagos por cliente

0..1 operación por pago
0..N pagos por operación

Controles: CHECK monto > 0; estados limitados; índice único método + número de operación.

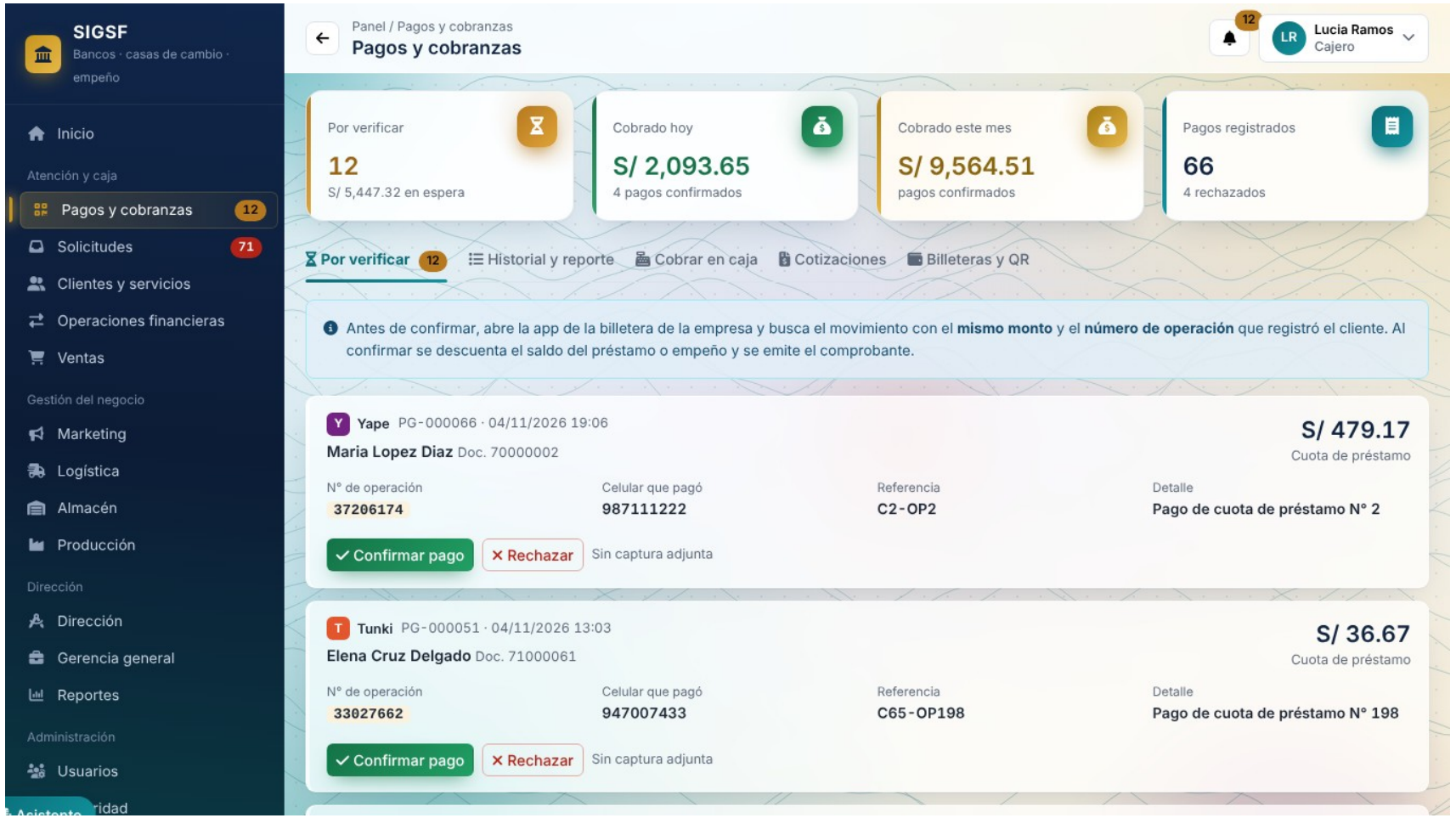
DDL entregado: 37 tablas únicas y 33 referencias FK. Sin introspección de una BD en ejecución.

Anexo 5. Diccionario de datos

Campo de pago	Tipo / nulabilidad	Regla
id_pago	SERIAL / NOT NULL	Clave primaria
id_cliente	INTEGER / NOT NULL	FK a cliente.id_cliente
id_operacion	INTEGER / NULL	FK a operacion; ON DELETE SET NULL
monto	NUMERIC(12,2) / NOT NULL	CHECK monto > 0
estado	VARCHAR(15) / NOT NULL	POR_VERIFICAR, CONFIRMADO, RECHAZADO
numero_operacion	VARCHAR(30) / NULL	Único con método, si no es NULL

Diccionario_DDL.csv contiene 287 atributos de 37 tablas con tipo, nulabilidad, claves, defecto y fuente.

Anexo 6. Prototipos UX/UI implementados





Pantallas recibidas con el proyecto; datos de demostración. No son nuevas capturas de esta revisión.

Anexo 7. Reportes y denominación

Reporte / documento	Denominación	Salida / destinatario
Registro de pago	PG-%06d	Portal y caja
Comprobante de pago confirmado	RC-%06d	HTML imprimible / PDF del navegador
Cotización	COT-%06d	HTML imprimible / PDF del navegador
Estado de cuenta	Por cliente	Portal: deuda y pagos
Pagos y cobranzas	Por estado / fecha	Pantalla y exportación
Exportaciones por recurso	Clientes, ventas, pagos y otros	XLSX, CSV y JSON según catálogo

Numeración confirmada en model/pagos/Pago.java y Cotizacion.java.

Anexo 8. Recursos del código fuente

Recurso	Implementación	Evidencia
Paquetes	controller, service, model, dto, dao, security	217 archivos Java en src/
Dependencias Java	PostgreSQL JDBC; SLF4J 2.0.16; Logback 1.5.12	lib/ y src/logback.xml
Frontend	HTML, JS, Bootstrap 5.3.3 local	web/vendor/ y api.js
Logback	Acceso, seguridad, errores y archivos por área	MDC, rotación y limpieza de saltos
Importación	CSV/XLSX: vista previa y validación por fila	ArchivoService y LectorExcel
Protecciones de ficheros	Sin DTD/XXE; límites ZIP; fórmulas neutralizadas	archivos/ y sus pruebas

Dependencias verificadas por archivos del ZIP. El análisis de vulnerabilidades de componentes sigue pendiente.

Anexo 9. Repositorio y versionamiento

Evidencia local

Base: v5.5.0 (bae0966)
Integración: v5.6.0
Rama de entrega: release/5.6.0
Rama de trabajo: feature/
integracion-seguridad-5.6.0

Merges y rollback

a453fe1 integra seguridad y pruebas.
614f35b ajusta la entrega.
Merge local y etiqueta v5.6.0.
Historial previo conservado.

Revisión y rollback de la entrega

```
git log --graph --oneline --decorate --all
git diff v5.5.0..v5.6.0 --stat
git switch -c rollback/5.6.0 release/5.6.0
git revert -m 1 v5.6.0   # propuesta para revertir el merge
```

Repositorio declarado: github.com/MiguelWaa/SistemaFinanciero. Disponibilidad remota no verificada.

Anexo 10. Pruebas de software y entorno

Nivel	Preparación / caso	Resultado o condición
Unitarias	Servicios con DAO en memoria	Incluidas en 214 JUnit
Integración	API, permisos y colección Postman	26 peticiones / 63 aserciones
Sistema / aceptación	Flujos de pago e historias de usuario	JUnit aprobado; navegador pendiente
Rendimiento local	500 peticiones, 20 usuarios del test	0 errores, p95: ver registro
Integración PostgreSQL	7 casos JDBC sobre una BD nueva	Código listo, falta PostgreSQL
Ejecución reproducible	JDK 17+, Node y Python 3	validar_proyecto.py

Resultado actual: 214 JUnit. Las 7 pruebas JDBC y las 14 E2E no se suman como aprobadas.

Anexo 11. Inventario y trazabilidad

Requisito	Documento / artefacto entregado
1. Procesos / 2. Patrones	docs/modelos/ y documentación de arquitectura
3. Clases / 4. Base de datos	Catálogo de 217 tipos Java, modelo ER y DDL
5. Diccionario / 6. UX/UI	287 atributos de 37 tablas y pantallas del sistema
7. Reportes / 8. Recursos	Reportes, recursos y documentación integral
9. Seguridad Java y SCV	Plan de seguridad, cambios de código y repositorio local
10. Pruebas y herramientas	Ejecutores, regresiones y evidencia actual

Anexo 12. Rutas compartidas y fuentes

JUnit 4

junit.org/junit4/

Newman / Postman

learning.postman.com/docs/…

Playwright

playwright.dev/docs/test-cli

OWASP WSTG

wstg.owasp.org/

ZAP Baseline

www.zaproxy.org/docs/docker/baseline-scan/

Dependency-Check

owasp.org/projects/dependency-check

Repositorio del proyecto

github.com/MiguelWaa/SistemaFinanciero

Pruebas / Actions

github.com/MiguelWaa/SistemaFinanciero/actions

Enlaces clicables. Las rutas del proyecto requieren permisos y la versión publicada correspondiente.